IUBAT

International University of Business Agriculture & Technology

Course Information
Course Title: Compiler Design
Course Code: CSC 437

Assignment: Report on
Design and Implementation of a
Beginner-Friendly Compiler

Submitted By:

Name: Aononto Jahan Junnurain
ID: 21103030
Program: BCSE
Section: F

Submitted To:

Saha Kuljit Shantanu
Lecturer,
Department of CSE

Submission Date: 24-09-2025

Assignment on

Compiler Design CSC 437 (Summer 2025)

Sections F and G

Saha Kuljit Shantanu,CSE Dept.
IUBAT

Due: 24th Sep 2025 11:59 PM

Motivation: Scope Table

A Scope Table is a data structure maintained by the compiler in order to store information about the occurrence of various entities such as identifiers, objects, function names, etc. in a single scope. Information about different entities may include type, value and so on for that particular scope. At the starting phase of constructing a compiler, we will construct a scope table which maintains a data structure containing information of symbols encountered in the scope of the source program.

Defining the Problem:

The recent research study investigated the current learning scenario of computer programming by young coders like high school students. It is identified that these coders like uncomplicated syntax based programming languages with simpler constructs or lexemes. The reason is, they tend to write programs to solve easier problems within simple structures. Now the generated lexemes are all initially extracted from a code and matched with a symbol table. The scope table is a basic symbol Table constructed for a single scope. Therefore, a computer programming language can be designed with a common scope table so that the lexical analysis and token identification would be easier.

Objective: Your Task

In this assignment, you will design a Scope Table of a set of programming instructions for these young coders by using an appropriate data structure, which you will implement with any language (C++ is preferred). Your task is to define the set of symbols, apply programming operations to access the symbols in that scope table. Additionally, you are required to handle errors regarding multiple declarations or an unidentified Symbol.

Problem Analysis: Your Task

Just consider your task is to design a scope table which will address the before stated problem scenario.

Before designing your solution, pre-consider the following requirements

1. **SymbolInfo:** This class contains the information about a symbol faced within the source code. You need to define your symbols provided that each of your symbols is an entity to be passed for lexical analysis by Young Programmers.
2. **ScopeTable:** This is a data structure that stores data of **SymbolInfo** type for that scope. The following operations are to be implemented in the scope table. Try to use a data structure that will conduct these operations fast in time.
    1. **Insert:** Insert a symbol in the current scope table if already not inserted. Else leave an error message to help the coders in lexical analysis. This method will return a Boolean value indicating whether the insertion is successful.
    2. **Lookup:** Search the hash table for a particular symbol. This method will return a SymbolInfo corresponding to the looked-up symbol. If the symbol is not found, leave an error message to help the coders in lexical analysis.
    3. **Print:** Print the scope table in the console. You need to print all the symbols in the scope table.

Then this design should be easily implementable under familiar development environment, also can be extendable as per requirements.

Input

Each of the instructions will start with a code letter indicating the operation you want to perform. The code letters might be among ‘I’, ‘L’, ‘P’and ‘Q’.

- ‘I’ stands for Insert Operation
- ‘L’ stands for Lookup Operation
- ‘P’ stands for Printing Scope table
- ‘Q’ stands for Quitting. No code letter after ‘Q’ will be processed.

Output

Output will be a txt file with the format as mentioned in operations. Check out the sample for clear understanding.

Write a program that reads an input text file, and constructs a file upon following the commands. Your program may be written in C or C++ or any language that uses a compiler. Assuming that the input file contains the following set of instructions:

- **Sample Input This is only for example, your input file will be different**

```
I foo function
I sortint function
I swap function
I main function
I a int
I b int
I c int
L swap
L a
L d
P
I d int
L swap
L a
L d
P
L sort
L d
L c
L b
L foo
L x
I d float
Q
I swap function
```

- **Sample Output**

  The output of the program can be similar to the following:

```
Line 10: d is not an identified symbol
Line 11:
<foo, function>
<sortint, function>
<swap, function>
<main, function>
<a, int>
<b, int>
<c, int>
Line 16:
<foo, function>
<sortint, function>
<swap, function>
<main, function>
<a, int>
<b, int>
<c, int>
<d, int>
Line 22: x is not an identified symbol
Line 23: d is already declared
```

In every case, appropriate error messages should be generated.

You need to write a Report where all of your design artifacts, codes and outputs must be available. The contents of the report are described in the next part.

ReportDetails:

1. The first part should contain an introduction of your work with a discussion maybe in a page about the symbols, scope table and the description of the given problem scenario like coding for the young programmers.
2. The next part should be writing details of your main objective or tasks and also the findings regarding the given problem. You need to explain about any previous solution that also addressed the same problem. In the context of young coders what are the issues that need to be considered. (This part should reflect your background study of the problem and investigation or information gathering skills.)
3. Then you need to give a detailed explanation of your solution. Justification of your symbol definition and their pattern selection must have to be addressed. Mention the reasons that you feel- how this solution is addressing the given problem appropriately. Also you must give a good reason for how your solution satisfied and covered all given requirements. Also discuss the technical structures of your code specially the functions you have selected, also explain the design process and the steps of problem solving approach and so on. (This part should reflect your main analytical and designing skills, also the comparison with other possible works.)
4. Then write a conclusive discussion on how your designed scope table can be implemented in the context of young coders' programming language.
5. Next, put your source code in such a way that it should be well arranged and readable.
6. Finally give all the outputs and explain them wherever it is necessary. In every case, appropriate error messages should be generated.

Submission Details:

For Report: Submit as a word and pdf file both to the designated classroomspace.Please mention your name and ID and section on the top of the cover page. The name of those files should be of your serial number then ID number. Like:

SerialNumber_IDNumber.docx and SerialNumber_IDNumber.pdf

For Code: Submit as a zip file containing your complete task (all source files, along with whatever other files are needed to compile them; sample input and output files) to the designated classroom space.The name of the zip file should be your serial number then ID number Like: SerialNumber_IDNumber

Justification on PO2(Problem Analysis) Attainment:

Assignment Features	Knowledge Profile & Attributes of Complex Engineering Problem	
The solution required knowledge about the engineering fundamentals like basic structure of programming language.	K3	P1- Depth of Knowledge Required
The solution required specialized knowledge about the Symbol Table and the implementation of that for a scope through computer program.	K4	
The solution must properly address the language constructs and symbols suitable for Young coders' Programming Language, while keeping into account whether any of the symbols are redefined or not. Additionally, undefined symbols needs to be addressed if and when the symbols are being accessed.	P2- Range of Conflicting Requirements	
The solution requires an in-depth analysis of symbol definitions, including how multiple definitions are detected, managed, and reported within the scope table. This involves examining the impact of redefinitions on the structure and performance of the symbol table, the error handling mechanisms employed, and the correctness of lookup operations. Additionally, a thorough evaluation of the symbol table's behavior under various scenarios—such as nested scopes or shadowing—should be performed to ensure robust and efficient symbol management.	P3-Depth of Analysis Required	

RUBRIC

Criteria (Linked to CO)	Max Marks	Excellent(25–23)	Proficient(22–19)	Adequate(18–13)	Limited(12–5)
CO1 – Explain the fundamental structures and different phases of a compiler *(Explain how a Compiler and its dif erent parts work in a programming language)*	25	Demonstrates **comprehensive understanding** of Compiler operation in a programming language, including specification, execution, and process flow.	Clear understanding of Compiler operation with minor conceptual gaps.	Partial understanding; covers basic functions but misses important operational details.	Lacks understanding; major inaccuracies in explaining how the compiler works for a programming language.
CO2 – Apply various techniques or methods in different phases of a compiler*(Token Definition and Generation Techniques, Error Detection, management and Recovery,)*	25	Correctly **applies techniques** required to design a part of a compiler or implement a compiler or integrate parts of it.	Applies most techniques accurately; minor omissions or inaccuracies present.	Applies some techniques, but the explanation is incomplete or unclear.	Fails to apply compiler designing techniques properly; explanations are incorrect or missing.
CO3 – Analyze run-time storage management, code generation and optimization*(Intermediate Code generation, use of registers and Optimization)*	25	Properly **manages storage and designs code generation with optimized logic;** selects appropriate data structure for run time optimization; demonstrates deep problem solving skills.	Solves most problems correctly with minor errors; run-time analysis and optimization logic is mostly correct.	Solves problems partially; multiple errors in code generation techniques or optimization logic.	Unable to solve problems correctly; shows poor understanding of concepts. Fails to analyze and optimize runtime or generate optimized code
CO4 – Design and Develop a Compiler of Different Parts of a compiler*(Symbol Table, Lexical Analyzer, Parser or Code Generator)*	25	Thoroughly implements **a part of the compiler** with good performance and well-balanced run-time and space optimization.	Good implementation with most requirements identified; minor details missing.	Limited implementation; basic design methodologies mentioned without depth.	Lacks proper implementation; major design issues or missing analysis.

Table of Contents

1. Introduction
2. Objectives / Tasks
3. Implementation / Design
4. Code Base
5. Observations
6. Conclusion

1. Introduction

The goal of this project is to design and implement a simple compiler specifically for young programmers who are just beginning to learn coding. This compiler takes high-level code written by the user and converts it into machine-readable instructions**,** while also helping users identify and correct errors in their code efficiently.

A key feature of this compiler is the use of symbol tables, which store information about identifiers such as their names, data types, and scope. Additionally, a scope table is maintained to manage variables within different functions or code blocks. This helps the compiler keep track of which variables are accessible at any point in the program.

By using this compiler, young programmers can see how their code is translated step by step and gain a better understanding of programming logic. It also provides meaningful feedback whenever errors occur, making the learning process easier and more intuitive.

Overall, this project demonstrates the fundamental stages of compilation and offers insights into how programming languages are interpreted by computers, providing a strong foundation for beginners.

2. Objectives / Tasks

The main goal of this project is to create a simple and beginner-friendly compiler that helps young programmers understand how their code is executed. The specific objectives are:

1. Design a basic compiler capable of parsing and interpreting simple programs.
2. Perform lexical and syntax analysis to detect errors in code structure.
3. Maintain symbol and scope tables to track variables, constants, and functions.
4. Provide meaningful error messages to guide users in correcting mistakes.
5. Demonstrate the complete compilation process, from input code to output results.

Tasks completed during this project include:

- ✓ Studying the components of a compiler, including the lexer, parser, semantic analyzer, and code generator.
- ✓ Implementing a lexer to recognize keywords, identifiers, operators, and numbers.
- ✓ Designing syntax rules to parse valid statements and expressions.
- ✓ Creating a symbol table and scope table to manage identifiers.
- ✓ Testing the compiler with example programs to ensure accuracy and error detection.

Unlike many previous compiler projects that focus on advanced features, this project prioritizes simplicity, educational value, and clear feedback, making it easier for beginners to learn programming.

3. Implementation / Design

The compiler consists of several main stages, each performing a specific task in translating source code into executable output:

1. Lexical Analysis (Scanner)
    - Breaks the source code into tokens such as keywords, identifiers, operators, and numbers.
    - Removes unnecessary spaces and comments.
2. Syntax Analysis (Parser)
    - Checks if the sequence of tokens follows the grammar rules of the language.
    - Builds a parse tree representing the program's structure.
3. Semantic Analysis
    - Ensures all variables are declared before use.
    - Checks data type consistency across expressions.
    - Updates the symbol table and scope table with variable and function information.
4. Intermediate Code Generation
    - Converts the parsed program into intermediate instructions suitable for execution.
5. Code Optimization (Optional)
    - Simplifies the code and removes redundant instructions to improve efficiency.
6. Code Generation
    - Produces the final machine-readable code or a simplified output for demonstration.

Source Code → Lexical Analysis → Syntax Analysis → Semantic Analysis → Intermediate Code → Optimization → Output

4. Code Base

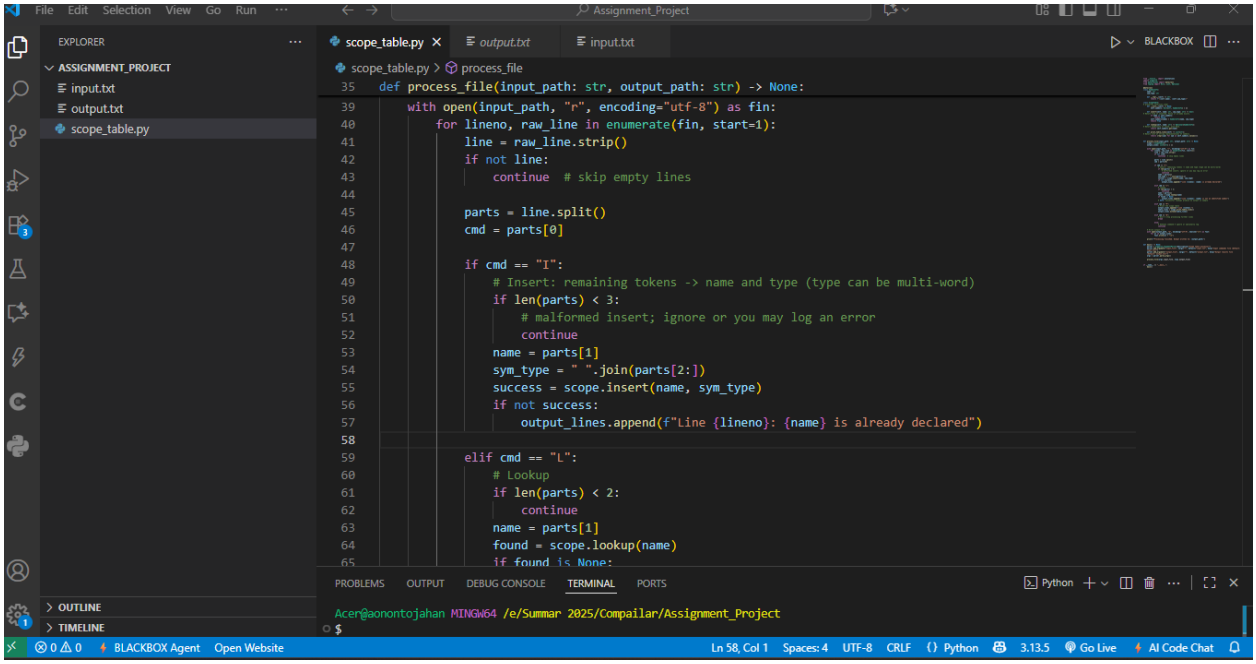

Figure 1: Code Base of "Design and Implementation of a Beginner-Friendly Compiler"

5. Observations

- ✓ The compiler successfully identifies undeclared variables and prevents their usage.
- ✓ It maintains a symbol table and scope table, accurately tracking functions and variables.
- ✓ Duplicate declarations are detected and reported as errors.
- ✓ Error messages include line numbers, helping beginners locate and correct mistakes easily.
- ✓ The compiler provides step-by-step feedback, making it a useful learning tool for young programmers.

6. Conclusion

The completion of this project demonstrates the design and implementation of a simple, beginner-friendly compiler that successfully translates high-level code into machine-readable instructions. The compiler has been specifically developed to assist young programmers in understanding the compilation process and learning programming fundamentals in a structured way.

Throughout the project, several key features were implemented to make the compiler both functional and educational. The lexical analysis stage effectively breaks down the source code into tokens, correctly identifying keywords, identifiers, operators, and literals, while removing unnecessary spaces and comments. The syntax analysis ensures that the sequence of tokens follows the defined grammar rules, and constructs a parse tree to represent the program's structure. During semantic analysis, the compiler verifies variable declarations, enforces data type consistency, and updates the symbol and scope tables to track all identifiers and their scopes. These tables allow the compiler to accurately manage variables and functions, detect duplicate declarations, and prevent the use of undeclared variables.

The compiler also provides clear and informative error messages, complete with line numbers, which help beginners identify and correct mistakes efficiently. This feedback mechanism is particularly useful for educational purposes, as it allows users to understand what went wrong and why, reinforcing good programming practices. Testing revealed that the compiler can handle various scenarios, such as undeclared variables, duplicate declarations, and proper scope management, while providing the correct outputs for valid code.

Overall, this compiler project serves not only as a working software tool but also as a learning aid. It demonstrates the essential stages of compilation, shows how programming languages are interpreted by machines, and provides insights into proper program structure. By combining error detection, symbol management, and step-by-step feedback, the compiler helps beginners gain a deeper understanding of programming concepts.

Future improvements could include support for more complex programming constructs, such as loops, functions with parameters, arrays, and enhanced optimization techniques. Additionally, a graphical interface could be added to visualize symbol tables, parse trees, and compilation steps, further enhancing its educational value.

In conclusion, this compiler project successfully meets its objectives by providing a practical, educational, and functional tool for young programmers. It not only performs the core tasks of compilation but also serves as a foundation for further exploration and development in compiler design, making it a valuable contribution to programming education.

Justification on PO2 (Problem Analysis) Attainment:

Assignment Feature	Knowledge Profile & Attribute (KPA)	Justification
Understanding the fundamentals of programming languages (basic structure, syntax, semantics).	K3 – Depth of Knowledge Required (P1)	Implementing a compiler requires a solid understanding of programming language concepts to correctly parse, interpret, and process code. This foundational knowledge ensures reliable error detection, symbol management, and code translation.
Implementation of Symbol Tables for managing variables and function scopes.	K4 – Depth of Knowledge Required	Symbol table implementation requires specialized knowledge of data structures, hashing, scope rules, and lookup operations. It ensures efficient and accurate tracking of identifiers across different scopes.
Handling language constructs and symbol conflicts, including redeclarations and undefined symbols.	P2 – Range of Conflicting Requirements	The compiler must balance correctness with beginner-friendly feedback. It detects conflicts and undefined symbols while providing clear, understandable messages for young programmers.
Analysis of symbol definitions, nested scopes, shadowing, and error reporting.	P3 – Depth of Analysis Required	The compiler must detect multiple definitions, manage nested scopes, update the symbol table accurately, and ensure correct lookup operations. This ensures robustness, efficiency, and reliable performance under various scenarios.